

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Fundamentación de Robótica	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez	
Módulo:	Actividad: Reporte Reto semanal 2 Manchester Robotics	
Fecha: 23 de febrero de 2026		

Resumen

En este proyecto se desarrolló un sistema de control para un motor de corriente directa en ROS2, donde se utilizó también la herramienta de `rqt_plot` y `rqt_graph` para observar el comportamiento de la simulación. Para ello se utilizó una referencia y con ayuda del controlador PID se busca acercarse lo más posible, esto por medio de los ajustes necesarios de los valores PID. Es importante mencionar que igualmente se utilizó Simulink en MATLAB, pues permitió un ajuste más acertado para los valores de las ganancias del controlador.

Objetivos

El objetivo principal del segundo reto fue integrar un controlador PID en ROS2, esto con el fin de controlar el comportamiento de un sistema donde hay un motor DC. Todo de manera simulada y con ayuda de un nodo de control en lazo cerrado para ajustar los parámetros PID en tiempo real. Este nodo de control se suscribe a la señal dada por `set_point` y publica la señal hacia el nodo del motor, `motor_sys`. Igualmente se utilizó un archivo `launch`, el cual facilita la ejecución de los nodos y permite la interacción con el `rqt_plot` donde se podrán observar las diferentes señales.

Introducción

El framework ROS o también conocido como Robot Operating System es un tipo de framework muy valioso para el desarrollo de aplicaciones para robots. En el presente reporte se describe el desarrollo de un mini reto enfocado en el diseño e implementación de un controlador digital para un sistema simulado dentro de ROS. Por ende, resulta importante describir los conceptos fundamentales utilizados en este reto.

En ROS2 se introdujeron los namespaces con el fin de mejorar la compatibilidad con sistemas multi robot. Por ende, resulta importante definir que es un namespace en ROS2. Un namespace permite crear un contexto único para los nodos, tópicos y servicios de ROS (Ramachandran, 2023). Es decir, permite que los nodos de ROS tengan el mismo nombre pero operen en contextos diferentes. Un ejemplo de lo anterior es cuando dos robots en la misma red pueden tener nodos y temas con el mismo nombre pero los namespaces les permiten operar de forma independiente. Es por ello que los namespaces son de mucha utilidad, ya que permite que se puedan replicar nombres y estructuras sin interferencias entre ellas.

Asimismo, en esta actividad se emplean los parameters o conocidos en español como parámetros, los cuales son valores de configuración que están asociados a los nodos. Los parámetros se utilizan para configurar externamente los nodos en tiempo de ejecución y durante el mismo. Es por ello que el tiempo de vida de un parámetro está ligado al tiempo de vida del nodo. Por ende, los parámetros pueden ser definidos como booleanos, enteros, flotantes o cadenas (Open Robotics, 2023). Esta funcionalidad en ROS es clave ya que nos permite ajustar características del sistema como las ganancias de un controlador sin necesidad de recompilar el código. De igual manera, para este mini reto aprendimos sobre los custom messages o conocidos en español como mensajes personalizados los cuales definen estructuras de datos específicas que permiten compartir información (Mathworks, s.f.)

Finalmente, este mini reto consistió en la implementación de un sistema de control para un motor. Nosotros implementamos un controlador proporcional integral derivativo (PID). Este tipo de controlador calcula la señal de control a partir del error entre el set point o la referencia deseada y la salida de nuestro sistema. El término proporcional permite que el sistema responda de manera inmediata al error actual, el integral permite eliminar el error y el término derivativo anticipa cambios en la dinámica de nuestro sistema al considerar la tasa de cambio en el error de la entrada. Es por ello que en este reto, la sintonización del controlador se realizó mediante el ajuste experimental en Simulink de las ganancias K_p , K_i y K_d . Estas ganancias fueron implementadas en nuestro nodo controller en ROS utilizando la ecuación

matemática dada por Manchester Robotics, lo que nos permitió evaluar el desempeño del sistema de control en lazo cerrado.

Es por ello que en este mini reto se integran conceptos de control, arquitectura de ROS2 y comunicación, lo que nos proporciona una visión de cómo se definen e implementan sistemas de control no solo en motores sino también en aplicaciones robóticas reales.

Solución del problema (Metodología)

Para resolver el mini reto se siguió una metodología dividida en cuatro etapas: análisis del sistema proporcionado, diseño del controlador, implementación de los nodos en ROS 2 y, finalmente, estructuración del lanzamiento con y sin namespaces

Metodología general:

Primero se revisaron las instrucciones del reto para identificar los nodos obligatorios, los tópicos y las restricciones (uso exclusivo de NumPy para las operaciones numéricas y tuning en tiempo de ejecución mediante parámetros).

Se analizó el nodo motorsys (simulación del motor DC) para entender sus parámetros (sampletime, sysgainK, systauT, initialconditions) y sus tópicos de entrada y salida (motor_input_u y motor_output_y).

Controlador PID diseñado en Simulink

Para el diseño inicial del controlador se implementó el diagrama de lazo cerrado en Simulink, donde la planta se modeló con la función de transferencia $G(s) = \frac{2.16}{0.05s + 1}$, conectada en serie con un bloque estándar *PID Controller* y realimentación unitaria. El bloque PID recibe la señal de error $e(t) = r(t) - y(t)$ y genera la señal de control $u(t)$ que alimenta a la planta, mientras que la salida $y(t)$ se envía de vuelta al sumador y al osciloscopio para observar la respuesta temporal. A partir de este esquema se buscaron ganancias que cumplieran con las especificaciones de sobreimpulso y tiempo de establecimiento; sin embargo, la herramienta de *Auto-Tune* del bloque PID no proporcionó resultados satisfactorios, por lo que se optó por un ajuste manual de las ganancias k_p , k_i y k_d mediante prueba y error, evaluando la respuesta ante cambios de referencia. De este proceso se obtuvieron los valores finales $k_p = 0.002454422522133$, $k_i = 2.35908845044266$ y $k_d = 0$, que posteriormente se trasladaron al controlador discreto implementado en ROS 2 como parámetros del nodo.

Descripción de los nodos y funciones

Con base en esto se definió la estructura de lazo cerrado:

- Nodo generador de referencia (set_point.py) publicando en el tópico setpoint.
- Nodo controlador (controller.py) recibiendo setpoint y motor_output_y, y publicando la señal de control en motor_input_u.
- Nodo de la planta (dc_motor.py) simulando la dinámica del motor DC.

El siguiente paso fue implementar el nodo de referencia sp_gen, encargado de generar diferentes tipos de señales de entrada para probar el controlador.

Se declararon parámetros configurables en tiempo de ejecución (amplitude, omega, timer_period, signal_type) y se implementó la lógica para generar tres tipos de señales:

- Senoidal: $u(t) = A \sin(\omega t)$
- Cuadrada: usando $\text{np.sign}(\text{np.sin}(\omega t))$ para forzar cambios bruscos en la referencia y evaluar la respuesta transitoria.
- Triangular: usando una combinación de np.sin y np.arcsin para construir una onda triangular a partir de una senoide.

La selección de la señal se hace a través del parámetro signal_type (sine, square, triangle), validando que el valor sea correcto y rechazando valores inválidos con mensajes de advertencia.

Además, al cambiar el timer_period, el nodo cancela el *timer* anterior y crea uno nuevo, garantizando que el periodo de muestreo del setpoint se actualice correctamente sin reiniciar el nodo.

Para el nodo de control ControllerNode se definió un controlador de tipo PID discreto en tiempo de muestreo T_s . Se declararon como parámetros los valores de T_s , k_p , k_i y k_d , de modo que pudieran ajustarse en ejecución usando `rqt_reconfigure`, cumpliendo con el requerimiento de tuning en tiempo real.

El nodo:

- Se suscribe al tópico setpoint para leer la referencia.
- Se suscribe al tópico motor_output_y para leer la velocidad/salida del motor.
- Publica la señal de control en el tópico motor_input_u que alimenta al nodo del motor.

En cada disparo del timer (período T_s) se ejecuta la ley de control:

1. Se calcula el error actual $e(k) = r(k) - y(k)$ a partir del setpoint y la salida medida.
2. Se actualiza el término integral acumulado $e(k) \cdot T_s$.
3. Se calcula el término derivativo como la diferencia de errores dividida entre T_s .
4. Se forma la salida de control $u(k) = K_p \cdot e(k) + k_i \sum_{n=0}^k e(k) + kd \frac{e(k) - e(k-1)}{T_s}$ y se publica en `motor_input_u`.

Para cumplir con la validación de parámetros, se implementó un *callback* de parámetros que verifica que T_s sea positivo y que k_p , k_i y k_d no sean negativos, rechazando configuraciones inválidas con mensajes de advertencia.

En el nodo `DCMotorNode` se usó el modelo de primer orden proporcionado, con parámetros configurables mediante parámetros ROS 2: tiempo de muestreo, ganancia K , constante de tiempo T y condición inicial. El *timer* de este nodo integra la ecuación en diferencias del sistema y publica la salida en el tópico `motor_output_y`, que alimenta al controlador.

Con esto se cerró el lazo de control: `setpoint` $\rightarrow$ `controller` $\rightarrow$ `motor_sys` $\rightarrow$ `motor_output_y` $\rightarrow$ `controller`, permitiendo observar la respuesta en `rqt_plot` y ajustar el PID en tiempo real.

Archivo de lanzamiento y seguimiento del código

Finalmente, se diseñaron dos archivos de lanzamiento para automatizar la ejecución de los nodos y el ajuste de parámetros.

1. Launch básico (`motor_launch.py`)
 - Lanza un grupo de tres nodos: `sp_gen` (setpoint), `controller` y `motor_sys`.
 - Define los parámetros principales de cada nodo (por ejemplo, `sampletime` y parámetros del PID) desde el propio launch, de manera que la configuración del experimento quede documentada y reproducible.
2. Launch con namespaces (`challenge_launch.py`)
 - Crea tres grupos de control independientes (`group1`, `group2`, `group3`) usando el parámetro `namespace` en cada Node.
 - En cada *namespace* se lanzan los tres nodos:
 - `motor_sys` con parámetros `sampletime`, `sysgainK`, `systauT` e `initialconditions`.

- `sp_gen` con parámetros `amplitude`, `omega`, `timer_period` y `signal_type`.
- `controller` con los parámetros `Ts`, `kp`, `ki` y `kd`.
- Se incluyen además los nodos de `rqt_plot` y `rqt_reconfigure` para visualizar las señales (`setpoint/data`, `motorinputu/data`, `motoroutputy/data`) y ajustar los parámetros en tiempo real.

El uso de *namespaces* permite instanciar tres lazos de control completos en paralelo, verificando con `rqt_graph` que cada grupo de tópicos y nodos se mantenga aislado (`/group1`, `/group2`, `/group3`), tal como se solicitó en el reto.

Resultados

El resultado de este *challenge* fue el diseño de dos archivos de lanzamiento y el diseño del controlador *PID* mediante una simulación de *Simulink* de *Mathworks*. Dicho sistema está representado de esta forma:

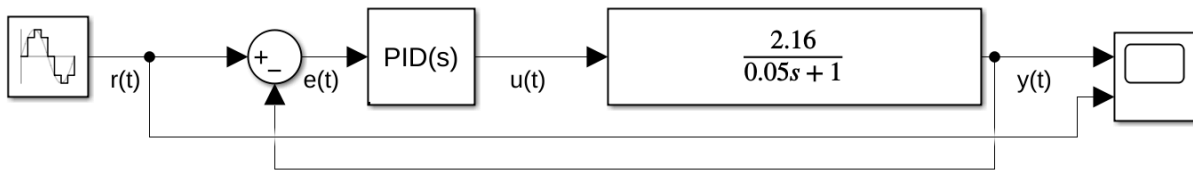


Imagen 1. Representación del controlador *PID* en bloques de *Simulink*.

Para la obtención de las ganancias k del controlador se utilizó la función *Tune* del mismo bloque. Al obtener la respuesta deseada, se ejecuta la simulación y se obtiene la siguiente respuesta:

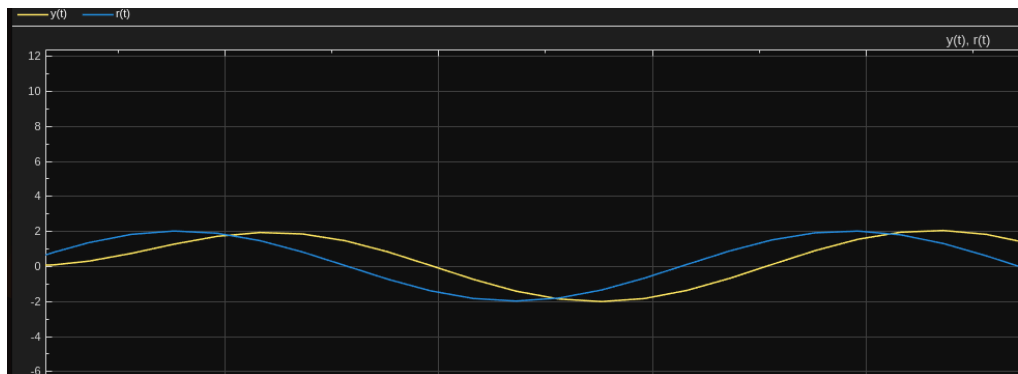
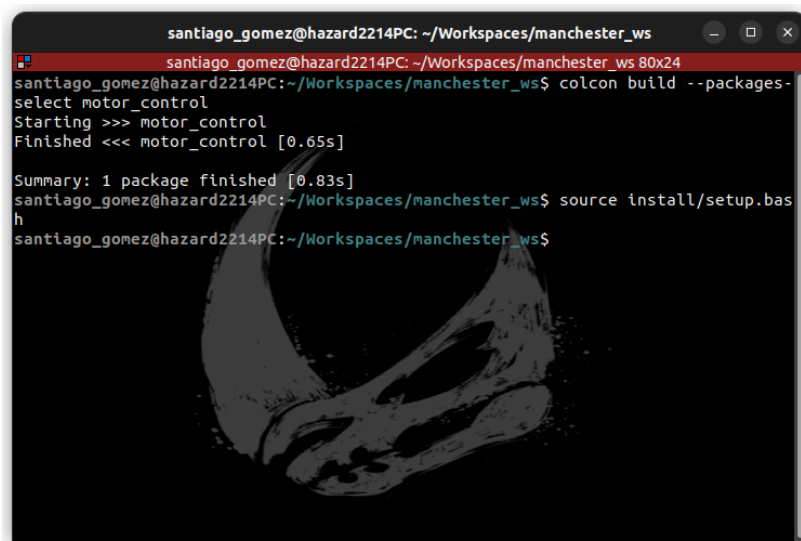


Imagen 2. Respuesta de la entrada y de la salida del sistema en *Simulink*.

Se puede observar que la salida del sistema se equipara en amplitud y frecuencia a la señal de entrada. Al haber concluido con el diseño del controlador, se prosiguió con el diseño del paquete de ROS 2.

Se compila el paquete *motor_control* en la carpeta del *Workspace* en el que está trabajando. Una vez realizado lo anterior, se configura el *workspace* para encontrar los paquetes y archivos necesarios para encontrar los archivos necesarios para el funcionamiento del paquete mediante el comando *source install/setup.bash*.

A terminal window titled 'santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws' with a red header bar. The terminal shows the following commands and output:

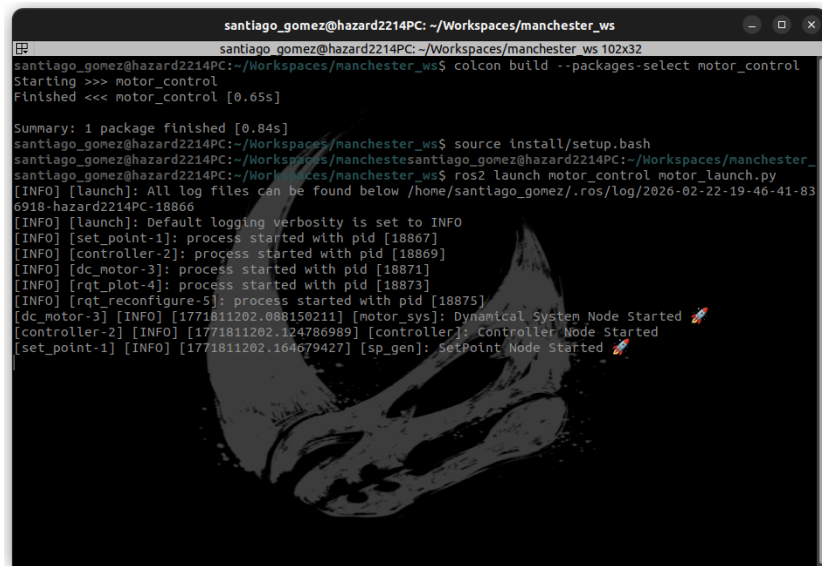
```
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws 80x24
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ colcon build --packages-
select motor_control
Starting >>> motor_control
Finished <<< motor_control [0.65s]

Summary: 1 package finished [0.83s]
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ source install/setup.bas
h
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$
```

The background of the terminal window features a faint, stylized image of a skull.

Imagen 3. Compilación del paquete y configuración del espacio del trabajo.

La primera parte de la implementación de este paquete es la aplicación del controlador *PID* en el sistema del motor DC. Tras la compilación del paquete y la configuración del espacio de trabajo, se ejecuta el primer archivo de lanzamiento mediante el comando *ros2 launch motor_control motor_launch.py*. Al ejecutarse, se lanzan mensajes tipo *log* que indican que los 3 nodos involucrados en el procesamiento (*sp_gen*, *controller* y *motor_sys*) han sido inicializados.



```
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ colcon build --packages-select motor_control
Starting >>> motor_control
Finished <<< motor_control [0.65s]

Summary: 1 package finished [0.84s]
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ source install/setup.bash
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 launch motor_control motor_launch.py
[INFO] [launch]: All log files can be found below /home/santiago_gomez/.ros/log/2026-02-22-19-46-41-83
6918-hazard2214PC-18866
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [set_point-1]: process started with pid [18867]
[INFO] [controller-2]: process started with pid [18869]
[INFO] [dc_motor-3]: process started with pid [18871]
[INFO] [rqt_plot-4]: process started with pid [18873]
[INFO] [rqt_reconfigure-5]: process started with pid [18875]
[dc_motor-3] [INFO] [1771811202.080150211] [motor_sys]: Dynamical System Node Started
[controller-2] [INFO] [1771811202.124786989] [controller]: Controller Node Started
[set_point-1] [INFO] [1771811202.164679427] [sp_gen]: SetPoint Node Started
```

Imagen 4. Ejecución del archivo de lanzamiento *motor_launch.py* en terminal.

Asimismo, el archivo de lanzamiento está configurado para ejecutar una ventana de *rqt_plot* y de *rqt_reconfigure*, del paquete *rqt*. La ventana de *rqt_plot* sirve para representar gráficamente el comportamiento de las señales publicadas en los siguientes tópicos:

- **/set_point/data:** Señal de entrada $r(t)$ proveniente del nodo *sp_gen*.
- **/motor_input_u/data:** Señal $u(t)$ que trabaja con las ganancias k (proporcional, integral y derivativa) y que también está afectada por la señal de entrada $r(t)$ y por el error $e(t)$. Proviene del nodo *controller*.
- **/motor_output_y/data:** Señal de salida $y(t)$ del sistema, proveniente del nodo *motor_sys*.

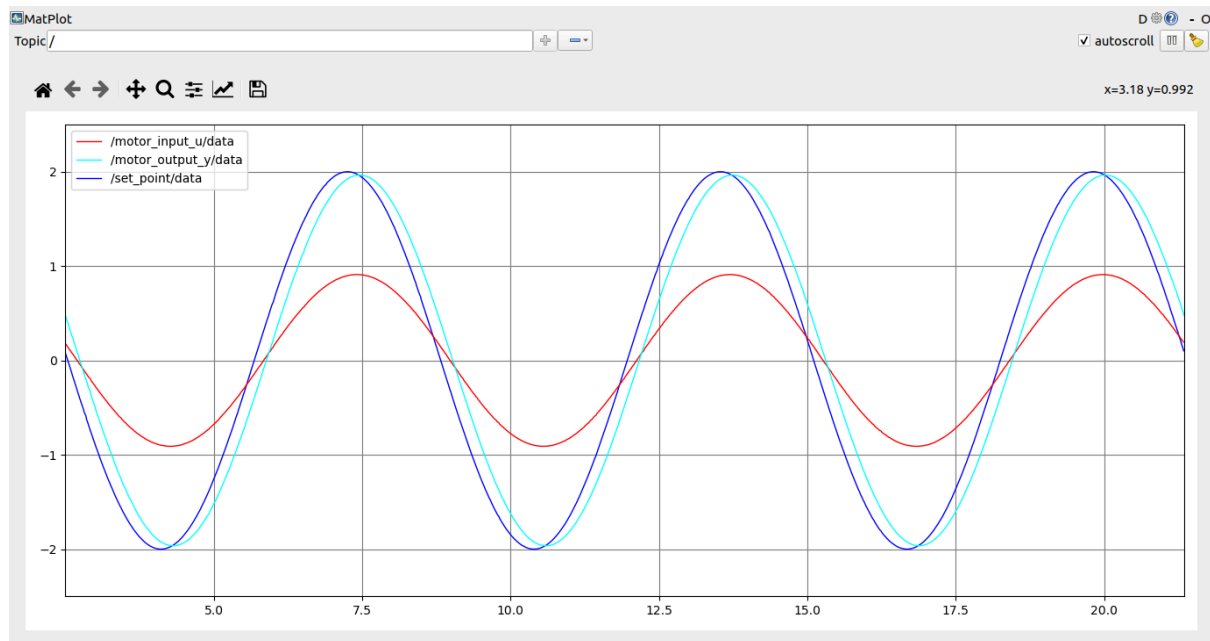


Imagen 5. Graficación de las señales dadas por los tópicos del sistema de control.

Nótese que inicialmente, la señal *set point* es una señal senoidal con una amplitud de 2 unidades y una frecuencia que simula una velocidad angular ω de 1 rad/s. La señal de salida graficada por del motor al final del sistema, iguala la amplitud y frecuencia de la señal original presentando un ligero desfase de la misma. Este desfase se debe a que el sistema modelado no es una ganancia estática, sino un sistema dinámico de primer orden. Al estar descrito por una ecuación diferencial, el sistema posee un polo real negativo que introduce retardo dinámico en la respuesta.

Desde el punto de vista del análisis en frecuencia, todo sistema de primer orden sometido a una entrada senoidal presenta un desfase dado por $\phi = -\tan^{-1}(\omega T)$ $\phi = -\tan$. Para una frecuencia de 1 rad/s, este término no es despreciable, lo que explica que la salida, aunque conserve la misma frecuencia y prácticamente la misma amplitud en régimen permanente, se encuentre temporalmente desplazada respecto a la referencia.

Para confirmar la interconexión de los nodos, se puede observar el comportamiento de los mismos mediante *rqt_graph*:

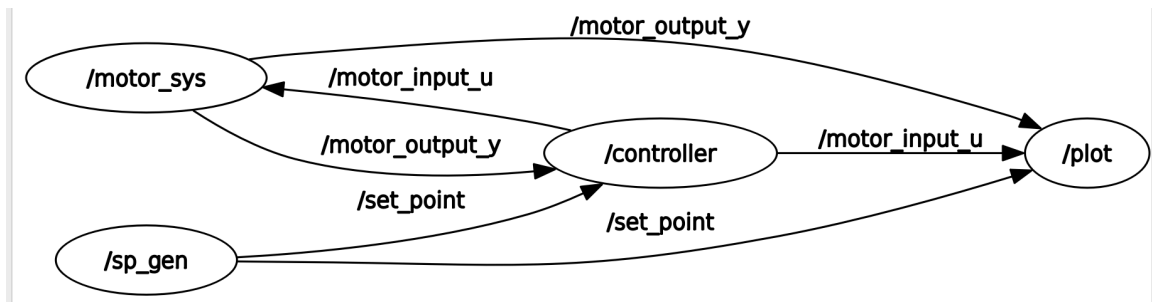


Imagen 6. Ilustración de la interconexión de los nodos utilizados.

La ventana de *rqt_configure* se utiliza para modificar en tiempo real el comportamiento de las tres señales al configurar el valor de los parámetros establecidos en cada nodo.

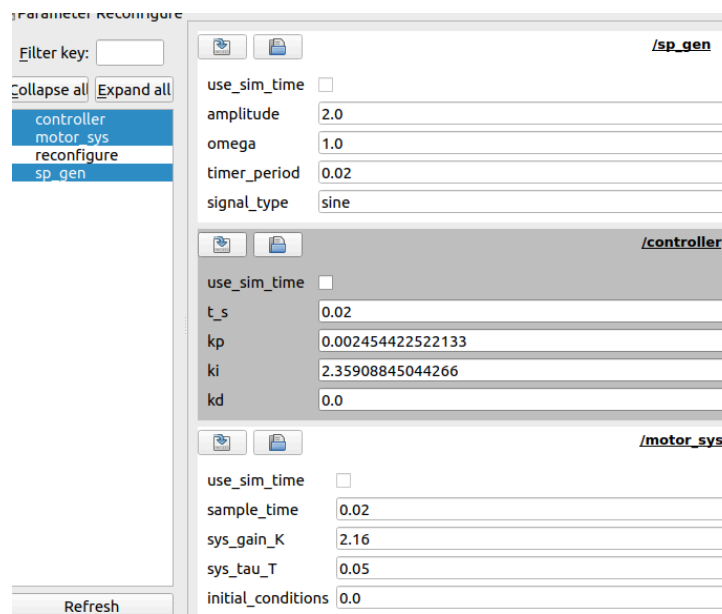


Imagen 7. Ventana de *rqt_reconfigure* con todos los parámetros a configurar.

En la imagen anterior, se puede apreciar que los parámetros que se pueden modificar incluyen el tiempo de muestreo de los tres nodos, la amplitud, la velocidad angular y el tipo de señal del nodo */sp_gen*, las ganancias *kp*, *ki* y *kd* de */controller* y los valores de *K* y τ en la función de transferencia $y[K + 1] = y[K] + ((-1/\tau)y[K] + (K/\tau)u[K])T_s$ que gobierna el comportamiento del motor.

Al alterar parámetros como las ganancias del controlador *PID*, el tipo de onda, la amplitud y el valor de τ , el resultado se vería de la siguiente forma:

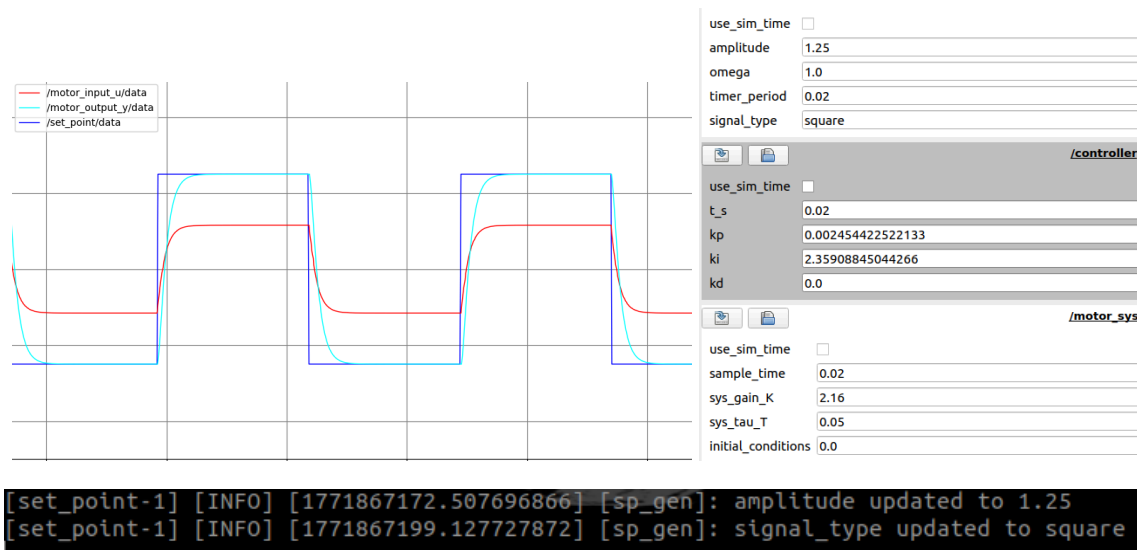


Imagen 8. Resultado de la reconfiguración de parámetros de amplitud y de tipo de onda.

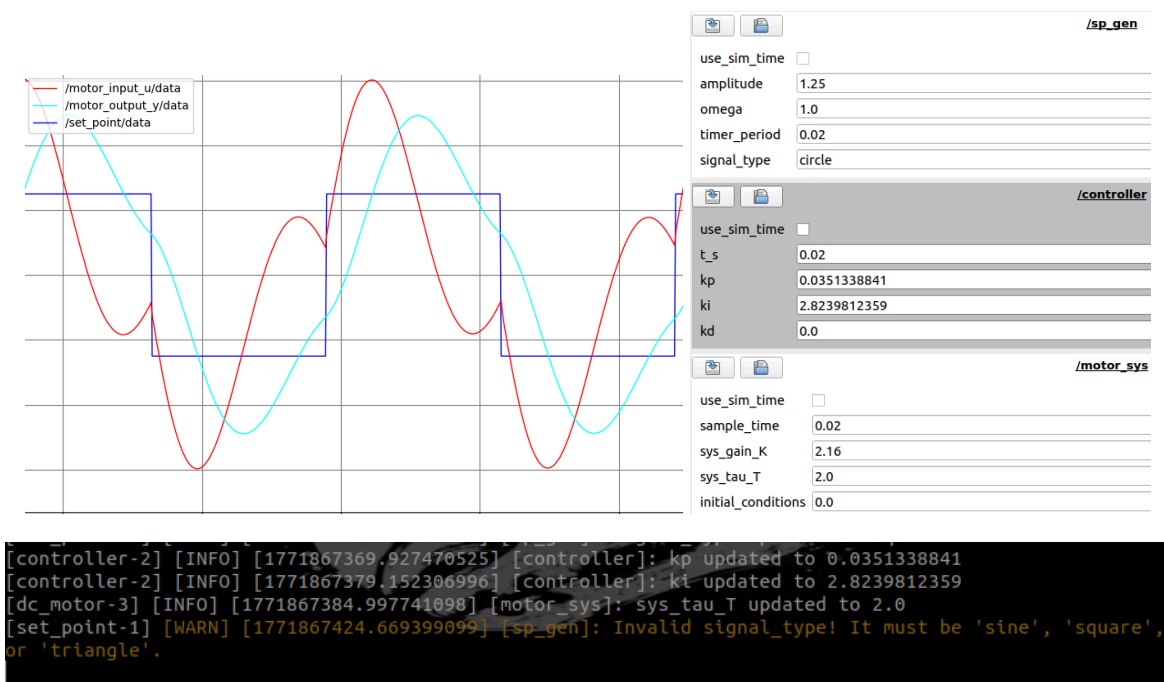


Imagen 9. Resultado de la reconfiguración de parámetros de ganancias k , τ , y tipo de onda inválida.

Se puede observar en la imagen anterior que el comportamiento de las 3 señales cambia al ajustar los parámetros. De igual manera, en terminal se despliegan mensajes de confirmación indicando a qué valores cambiaron ciertos parámetros, y proveyendo una advertencia en caso de ingresar un valor inválido en algún parámetro.

La segunda implementación en el presente *challenge* incluye la asignación de *namespaces* a distintos nodos. Para ello, se creó un segundo archivo de lanzamiento, llamado

challenge_launch.py, en el cual se crean tres grupos de sistemas de control de motor DC, cada uno bajo el *namespace* de *group1*, *group2* o *group3*. El resultado puede verse de la siguiente manera:

```

santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws
[INFO] [set_point-1]: process has finished cleanly [pid 18867]
[INFO] [rqt_plot-4]: process has finished cleanly [pid 18873]
[INFO] [rqt_reconfigure-5]: process has finished cleanly [pid 18875]
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 launch motor_control challenge_launch.py
[INFO] [launch]: All log files can be found below /home/santiago_gomez/.ros/log/2026-02-22-21-15-24-29
9834-hazard2214PC-21852
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [set_point-1]: process started with pid [21853]
[INFO] [controller-2]: process started with pid [21855]
[INFO] [dc_motor-3]: process started with pid [21857]
[INFO] [set_point-4]: process started with pid [21859]
[INFO] [controller-5]: process started with pid [21861]
[INFO] [dc_motor-6]: process started with pid [21863]
[INFO] [set_point-7]: process started with pid [21865]
[INFO] [controller-8]: process started with pid [21867]
[INFO] [dc_motor-9]: process started with pid [21869]
[dc_motor-6] [INFO] [1771816524.715569654] [group2.motor_sys]: Dynamical System Node Started
[dc_motor-9] [INFO] [1771816524.715193967] [group3.motor_sys]: Dynamical System Node Started
[set_point-4] [INFO] [1771816524.719382648] [group2.sp_gen]: SetPoint Node Started
[dc_motor-3] [INFO] [1771816524.726327518] [group1.motor_sys]: Dynamical System Node Started
[controller-2] [INFO] [1771816524.734376526] [group1.controller]: Controller Node Started
[set_point-1] [INFO] [1771816524.745984859] [group1.sp_gen]: SetPoint Node Started
[controller-5] [INFO] [1771816524.777738122] [group2.controller]: Controller Node Started
[set_point-7] [INFO] [1771816524.794764881] [group3.sp_gen]: SetPoint Node Started
[controller-8] [INFO] [1771816524.794958333] [group3.controller]: Controller Node Started

santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws 102x24
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 node list
/group1/controller
/group1/motor_sys
/group1/sp_gen
/group2/controller
/group2/motor_sys
/group2/sp_gen
/group3/controller
/group3/motor_sys
/group3/sp_gen
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 topic list
/group1/motor_input_u
/group1/motor_output_y
/group1/set_point
/group2/motor_input_u
/group2/motor_output_y
/group2/set_point
/group3/motor_input_u
/group3/motor_output_y
/group3/set_point
/parameter_events
/rosout
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$

```

Imagen 9. Resultado de lanzamiento del launch file *challenge_launch.py*, junto con los nodos y tópicos lanzados bajo su respectivo *namespace*.

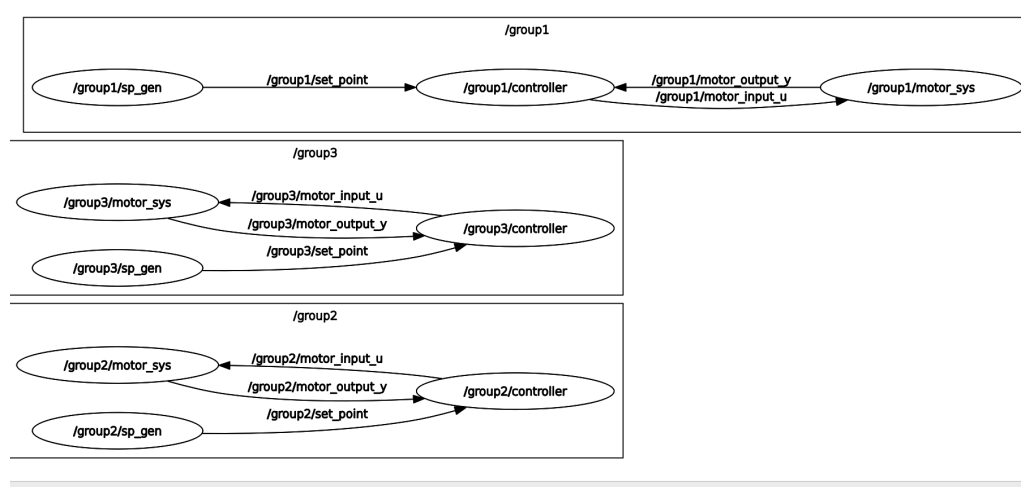


Imagen 10. Resultado de la interconexión de los nodos lanzados bajo su propio *namespace*.

Conclusiones

En conclusión, podemos decir que se logró implementar de manera satisfactoria un sistema de control en lazo cerrado en ROS2, integrando la comunicación entre nodos `sp_gen`, `controller` y `motor_sys`. Como se mostró con anterioridad, la correcta compilación del paquete, configuración del workspace y ejecución de archivo *launch* permitieron validar el funcionamiento correcto del sistema y de nuestros códigos.

En cuanto al cumplimiento de los objetivos, estos se alcanzaron de manera satisfactoria ya que el nodo controlador se implementó de manera correcta el algoritmo PID y fue capaz de regular la salida del sistema ante distintas señales de entrada generadas por el nodo `sp_gen`. Asimismo, a través de las gráficas obtenidas mediante `rqt_plot` pudimos verificar que la señal de salida sigue adecuadamente la referencia en amplitud y frecuencia, confirmando que las ganancias que introdujimos en nuestro código de ROS obtenidas a través de simulink fueron funcionales y correctas. De igual manera, el uso de herramientas como `rqt_graph` validó la comunicación correcta entre nodos y `rqt_reconfigure` permitió que se pudieran modificar parámetros en tiempo real, como lo fueron las ganancias del `controller` y el tipo de señal.

Es por ello que es importante mencionar que aunque el sistema logró un seguimiento correcto y adecuado de la señal de referencia, se observaron ligeras variaciones entre la señal de entrada y salida en forma de desfase. Este comportamiento es esperado ya que el motor introduce un tipo de retardo en la respuesta, además de que aunque se realizó el controlador en Simulink, la elección de las ganancias se fueron realizando un poco de manera heurística. Como posible mejora proponemos realizar un ajuste más detallado del controlador PID, empleando un método matemático para mejorar el desempeño del controlador y el sistema.

Referencias

Mathworks. (s.f.). Soporte de mensajes personalizados de ROS.

<https://www.mathworks.com/help/ros/ug/ros-custom-message-support.html>

Open Robotics. (2023). Acerca de los parámetros en ROS 2.

https://ros-spanish-users-group.github.io/ros2_documentation/eloquent/Concepts/About-ROS-2-Parameters.html

Ramachandran, R. (2023). Domain ID and Namespace in ROS 2 for Multi-Robot Systems.

<https://blog.robotair.io/domain-id-and-namespace-in-ros-2-for-multi-robot-systems-9a939ae3fa40>